S^3-Vote

Secure, Scalable, Sovereign

A Comparative Study of Social Secret Sharing in Decentralized Voting

---

Reputation-Weighted Adaptive Thresholds,
Formal Security Analysis, and
Blockchain-Anchored Accountability

Author: Sepideh Bakhshi

Neptune ID: L7LAM9

Supervisor: Dr. Ligeti Péter

Faculty: Faculty of Informatics

Date: May 10, 2026

Abstract

Secure electronic voting demands confidentiality of individual ballots, integrity of the final tally, public verifiability of correct execution, and resilience against colluding or misbehaving participants — all without relying on a single trusted authority. This thesis presents **S^3-Vote** (*Secure, Scalable, Sovereign Voting*), a decentralised voting protocol that integrates ElGamal threshold encryption, Shamir Secret Sharing, and Publicly Verifiable Secret Sharing (PVSS) with a novel *reputation-weighted adaptive reconstruction threshold*.

Each decryption authority accumulates a persistent reputation score derived from verifiable on-chain behaviour across elections, covering participation, contribution, and honesty. Reputation scores are converted into per-authority weights, and the effective quorum adapts via a two-piece linear formula anchored at $\lceil N/2 \rceil$ for neutral trust. A *two-gate reconstruction policy* simultaneously enforces a cryptographic share-count gate and a reputation weight gate, closing flooding and monopoly attacks that single-threshold schemes cannot prevent. A concentration-resistance property is proven: no single authority can accumulate sufficient weight to reconstruct alone.

Ballot privacy is formally proven as an IND-CPA security game with a concrete reduction to the Decisional Diffie-Hellman (DDH) assumption under the random oracle model. A game-theoretic analysis shows that honest participation is the dominant strategy for rational authorities, and that coalitions are inherently unstable. A parameter sensitivity analysis characterises the effect of each reputation weight on system behaviour and bounds the consequences of miscalibration.

All events are recorded on a hash-chained blockchain, providing a tamper-evident, non-repudiable audit trail. The system is compared against Shamir's threshold scheme, Nojoumian's social secret sharing, and deployed e-voting systems (FOO92, Helios, Belenios, Civitas) — none of which address adaptive thresholds, authority accountability, or graceful failure recovery simultaneously.

Contents

List of Symbols **6**

1 Introduction **7**
1.1 Motivation . . . 7
1.2 Problem Statement . . . 7
1.3 Contributions . . . 7
1.4 Thesis Structure . . . 8

2 Background and Related Work **8**
2.1 ElGamal Encryption . . . 8
2.2 Shamir Secret Sharing . . . 9
2.3 Publicly Verifiable Secret Sharing . . . 9
2.4 Blockchain Technology . . . 10
2.5 Prior Work on Reputation in Secret Sharing . . . 10
2.6 Electronic Voting Systems: State of the Art . . . 11

3 System Architecture **13**
3.1 Design Goals . . . 13
3.2 Threat Model . . . 14
3.3 System Participants and High-Level Architecture . . . 15
3.4 Election Lifecycle . . . 16

4 Reputation Model **16**
4.1 Behavioural Scoring . . . 16
4.2 Reputation Update Formula . . . 17
4.3 Initial Reputation . . . 17
4.4 Reputation Weights . . . 17
4.5 Reputation Evolution . . . 18
4.6 Persistence Across Elections . . . 18

5 Adaptive Threshold **18**
5.1 Trust Evenness . . . 18
5.2 Two-Piece Linear Adaptive Formula . . . 19
5.3 Quorum Range by Group Size . . . 20

6 Two-Gate Reconstruction Policy **20**
6.1 Formal Definition . . . 20
6.2 Why Two Gates Are Necessary . . . 21

6.3 Impossibility Detection 21

7 Blockchain Integration **22**
7.1 Block Structure 22
7.2 Events Recorded on the Blockchain 22
7.3 Non-Repudiation Guarantee 23
7.4 Production Deployment Note 23

8 Security Analysis **23**
8.1 Vote Confidentiality 23
8.2 Tally Integrity 23
8.3 Cheat Detection and Accountability 24
8.4 Sybil Resistance 24
8.5 Collusion Resistance 24
8.6 Concentration Attack 24
8.7 Formal Ballot Privacy Proof 25
8.8 Receipt-Freeness and Coercion Resistance 26
8.9 End-to-End Verifiability 27

9 Game-Theoretic Incentive Analysis **28**
9.1 Model 29
9.2 Cheating is a Dominated Strategy 29
9.3 Coalition Instability 30
9.4 Sybil Rationality Analysis 30

10 Parameter Sensitivity Analysis **30**
10.1 Effect of Memory Factor λ 31
10.2 Effect of Participation Weight α 31
10.3 Effect of Decay Factor δ 32
10.4 Sensitivity Summary 33

11 Comparison with Prior Work **33**
11.1 Against Shamir's Threshold Scheme 33
11.2 Against Nojoumian's Social Secret Sharing 34

12 Prototype Implementation **35**
12.1 Module Overview 35
12.2 Key Algorithms in Code 35
12.3 Web Dashboard 36

13 Example Walkthrough **37**
13.1 Round 1 — Initial State 37
13.2 Round 2 — Reputations Loaded from Registry 37

14 Limitations and Future Work **38**

15 Conclusion **38**

List of Figures

1 PVSS share creation and verification flow. Commitments are anchored on-chain before any share is submitted, preventing retroactive manipulation. 10
2 High-level S^3-Vote architecture. All cryptographic operations write immutable records to the blockchain. The reputation engine updates authority scores after each round and feeds them into the next election's threshold computation. 15
3 S^3-Vote election lifecycle. Five normal phases form a clockwise cycle (blue); impossible reconstruction transitions to a failed state, applies penalties, and re-enters the reputation update phase (red). 16
4 Reputation trajectories for three authority behavioural profiles across 10 election rounds. Honest authorities converge toward high reputation; a single cheating event causes a severe drop that requires many honest rounds to recover from; skipping causes a moderate penalty. 18
5 Adaptive quorum as a function of collective trust for three group sizes. All curves pass through $\tau_{\text{neutral}} = \lceil N/2 \rceil$ at the initial trust value of $\frac{2}{3}$ (vertical dashed line), confirming Theorem 5.1. 20
6 Two-gate reconstruction policy. Gate 1 enforces the cryptographic minimum share count; Gate 2 enforces the trust weight minimum. Failure of either gate blocks reconstruction and may trigger impossibility detection. . 21
7 Blockchain block structure. Each block's hash includes the previous block's hash, forming a chain. Altering any block's data changes its hash, invalidating all subsequent blocks. 22
8 Effect of memory factor λ on long-run reputation for an always-honest authority ($\delta = 0.95$ fixed). High λ causes decay to dominate: even perfect behaviour cannot prevent slow reputation erosion. Low λ converges fast but makes reputation volatile. The default $\lambda = 0.7$ balances stability and responsiveness. 31

9 Effect of participation weight α on reputation recovery after a single cheat event (round 1), followed by honest participation. Lower α produces a deeper initial penalty. All trajectories converge to the same long-run equilibrium (≈ 0.84) because the honest-participation score is 1.0 regardless of α. 32

10 Quorum evolution for $N = 7$ over 8 rounds. When all authorities behave honestly, the quorum steadily decreases toward the floor of 3. One round of cheating/skipping deflects the trajectory upward, requiring several honest rounds to recover. 38

List of Symbols

Symbol	**Meaning**
N	Number of decryption authorities
t	Shamir threshold (minimum shares for reconstruction)
τ	Effective quorum (reputation-weighted threshold)
τ_{neutral}	Neutral quorum $= \lceil N/2 \rceil$
$\tau_{\min}$	Minimum quorum $= \max(2, \lceil N/3 \rceil)$
$\tau_{\max}$	Maximum quorum $= N - 1$
R_i	Reputation score of authority A_i
$\bar{R}$	Mean reputation across all authorities
w_i	Reputation weight of authority A_i
E	Trust evenness factor
τ_{trust}	Combined trust score $= \min(1, \bar{R} \cdot E)$
α	Participation weight in reputation formula (default 0.2)
β	Contribution weight in reputation formula (default 0.3)
γ	Honesty weight in reputation formula (default 0.5)
λ	Memory factor — old vs. new reputation (default 0.7)
δ	Decay factor per round (default 0.95)
P_i, C_i, H_i	Participation, contribution, honesty scores for A_i
p	Large prime modulus for ElGamal group
g	Generator of cyclic group $\mathbb{Z}_p^*$
x	ElGamal private key
y	ElGamal public key $= g^x \bmod p$
k	Ephemeral encryption randomness
s_i	Shamir share of authority A_i
C_i	PVSS commitment for share s_i, defined as $g^{H(s_i)} \bmod p$
$\mathcal{S}$	Set of authorities who submitted valid shares
$\mathbb{G}$	Cyclic group of prime order q

1 Introduction

1.1 Motivation

The integrity of voting systems is foundational to democratic governance and organisational decision-making. Traditional electronic voting systems route all trust through a central authority: a government agency, corporate IT infrastructure, or a cloud provider. This design introduces two compounding vulnerabilities simultaneously.

First, a **single point of trust**: the operator has complete power to manipulate ballots, suppress results, or alter tallies, often without any publicly verifiable evidence of wrongdoing. Second, a **single point of failure**: if the operator's infrastructure is compromised — through malicious attack, system failure, or coercion — the entire election is ruined.

Distributed threshold cryptography was developed to address these concerns by splitting trust across multiple independent parties. Yet classical threshold schemes such as Shamir's foundational work [1] treat every participant as equally trustworthy and provide no mechanism for detecting, proving, or adapting to misbehaviour. A single malicious authority submitting a corrupted share causes silent reconstruction failure, with no accountability and no path to recovery.

1.2 Problem Statement

The core problem this thesis addresses is:

Central Question

How can we design a decentralised voting system where the reconstruction threshold adapts dynamically to observed authority behaviour, cheating is publicly provable rather than merely suspected, and the system recovers gracefully from partial failures — all without relying on any trusted coordinator?

1.3 Contributions

This thesis makes the following contributions:

C1. Reputation-weighted secret sharing. A persistent, cross-election reputation model for decryption authorities based on three objectively measurable behavioural dimensions.

C2. Two-piece linear adaptive threshold. A provably anchored quorum formula that maps collective trust to a reconstruction requirement between $\lceil N/3 \rceil$ and

$N - 1$, initialising at exactly $\lceil N/2 \rceil$ for fresh authorities.

C3. Two-gate reconstruction policy. Simultaneous enforcement of a cryptographic share-count gate and a reputation weight gate, closing attack vectors that single-threshold schemes cannot prevent.

C4. Concentration-resistance proof. Formal demonstration that no single authority can ever accumulate sufficient weight to satisfy the quorum alone, regardless of their reputation.

C5. End-to-end prototype. A fully operational implementation comprising ElGamal encryption, Shamir sharing, PVSS verification, reputation engine, adaptive threshold, and blockchain audit log, accessible through a real-time web dashboard.

C6. Formal ballot privacy proof. A concrete IND-CPA security reduction from the S^3-Vote threshold ElGamal scheme to the DDH problem, with PVSS binding established under the discrete logarithm assumption and random oracle model.

C7. Game-theoretic honesty dominance. A formal proof that honest participation strictly dominates cheating for rational authorities over any sufficiently long horizon, and that colluding coalitions are inherently unstable under the reputation penalty mechanism.

1.4 Thesis Structure

Section 2 reviews the cryptographic primitives, prior work, and the state of the art in e-voting systems. Section 3 presents the system architecture and threat model. Sections 4–7 detail the core technical contributions. Section 8 provides the security analysis, including a formal IND-CPA ballot privacy proof under DDH, a treatment of receipt-freeness, and end-to-end verifiability. Section 9 analyses incentives game-theoretically, proving that honest behaviour is dominant for rational authorities. Section 10 presents a parameter sensitivity analysis. Section 11 compares S^3-Vote against prior work. Section 12 describes the prototype. Section 13 traces a complete two-round example. Sections 14 and 15 discuss remaining limitations and conclude.

2 Background and Related Work

2.1 ElGamal Encryption

The ElGamal scheme [2] derives its security from the computational hardness of the discrete logarithm problem. All arithmetic takes place in the multiplicative group $\mathbb{Z}_p^*$,

where p is a large prime and g is a generator of that group.

Definition 2.1 (ElGamal Key Pair)**.** A key pair (y, x) is generated by choosing a random private key $x \in \mathbb{Z}_{p-1}$ and computing public key $y = g^x \bmod p$.

To encrypt a vote encoded as a group element $m \in \mathbb{Z}_p^*$:

$$C_1 = g^k \bmod p \tag{1}$$

$$C_2 = m \cdot y^k \bmod p \tag{2}$$

where k is a fresh random ephemeral value for each encryption. Decryption recovers $m = C_2 \cdot (C_1^x)^{-1} \bmod p$. The scheme is semantically secure under the Decisional Diffie-Hellman (DDH) assumption [2, 14].

In S^3-Vote, the private key x is never held whole by any single party; it is split using Shamir Secret Sharing before the election begins and reconstructed only after all ballots are cast.

2.2 Shamir Secret Sharing

Definition 2.2 ((t, n)-Threshold Scheme [1])**.** A secret $s \in \mathbb{Z}_p$ is split by constructing a random polynomial of degree $t-1$ with $f(0) = s$:

$$f(z) = s + a_1 z + a_2 z^2 + \cdots + a_{t-1} z^{t-1} \pmod p \tag{3}$$

Authority i receives share $s_i = f(i)$. Any t shares uniquely determine f via Lagrange interpolation:

$$s = f(0) = \sum_{i \in \mathcal{S}} s_i \prod_{j \in \mathcal{S}, j \neq i} \frac{-j}{i - j} \pmod p \tag{4}$$

Any $t - 1$ shares reveal no information about s (information-theoretic secrecy).

The classical scheme treats all shares as interchangeable: any t-subset of the n shares suffices for reconstruction. Desmedt and Frankel [8] extended this idea to threshold cryptosystems, enabling distributed decryption without reconstructing the private key in full — a precursor to the threshold ElGamal decryption used in S^3-Vote. S^3-Vote further augments this with reputation weights, so that the effective contribution of each authority to the reconstruction quorum depends on their trust level.

2.3 Publicly Verifiable Secret Sharing

In standard secret sharing, a participant who submits a corrupted share causes reconstruction failure with no way to publicly identify the culprit. PVSS [3, 6] solves this via

cryptographic *commitments* that allow anyone to verify share correctness without learning the secret.

Definition 2.3 (PVSS Commitment)**.** For share s_i and cryptographic hash H, the commitment is:

$$C_i = g^{H(s_i)} \bmod p \tag{5}$$

Commitments are published to the blockchain at election creation. During reconstruction, authority i submits $\hat{s}_i$; the verifier checks $g^{H(\hat{s}_i)} \equiv C_i \pmod p$. A mismatch constitutes publicly verifiable proof of an invalid submission.

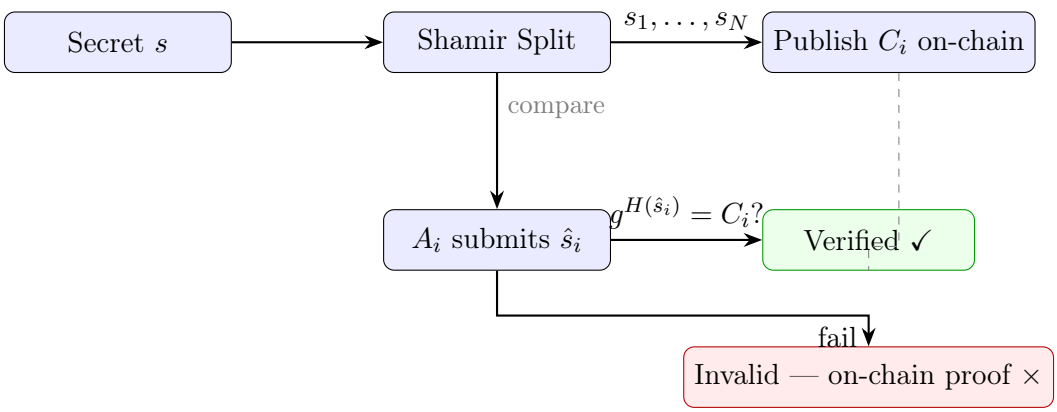

Figure 1: PVSS share creation and verification flow. Commitments are anchored on-chain before any share is submitted, preventing retroactive manipulation.

2.4 Blockchain Technology

A blockchain structures records as an ordered sequence of blocks where every block commits to the one before it via a cryptographic hash. Concretely, block B_k incorporates a hash of B_{k-1}:

$$h_k = \text{SHA-256}(\, k \,\|\, \text{timestamp}_k \,\|\, \text{data}_k \,\|\, h_{k-1} \,) \tag{6}$$

Any retroactive modification to block B_j invalidates every hash $h_j, h_{j+1}, \ldots$ — a mismatch that any replica can detect immediately, without trusting a central authority.

S^3-Vote uses the blockchain as a *public bulletin board*: all events are written once and cannot be silently revised.

2.5 Prior Work on Reputation in Secret Sharing

Nojoumian and Stinson [4,5] introduced *social secret sharing*, where participant reputation influences the effective threshold. Their key insight — that not all participants should

be treated equally and that past behaviour should inform future requirements — directly motivates S^3-Vote.

However, several critical limitations remain in their model: (i) a *trusted dealer* is required to assign and update reputation; (ii) reputation updates are social and subjective, opening them to coalition manipulation; (iii) no public proof of cheating is provided; (iv) a single threshold gate is used; (v) no protection against reputation concentration; (vi) no cross-round persistence mechanism is formalised; (vii) the work is framed as a secret sharing primitive, not a complete voting system. Section 11 provides a detailed per-point comparison. Cramer, Gennaro, and Schoenmakers [7] proposed an early multi-authority election scheme using threshold ElGamal and zero-knowledge proofs; S^3-Vote builds on this cryptographic foundation while adding reputation-weighted adaptive thresholds and cross-round accountability.

2.6 Electronic Voting Systems: State of the Art

Electronic voting has been studied since the early 1980s. We survey the most influential deployed systems and identify the gaps that S^3-Vote addresses.

FOO92

Fujioka, Okamoto, and Ohta [10] introduced the first widely-cited cryptographic e-voting protocol. FOO92 uses blind signatures to achieve anonymity: each voter obtains a blind signature from the administrator on their ballot and submits it anonymously through an untappable channel. The protocol achieves individual verifiability but relies on a *single trusted administrator* for signature issuance — a central point of failure that S^3-Vote eliminates through threshold key management. Furthermore, FOO92 provides no mechanism for handling a compromised or unresponsive administrator, and offers no accountability trail for administrator behaviour.

Helios

Helios [11] is an open-audit web-based voting system that uses ElGamal encryption and mix-nets or homomorphic tallying. Ballots are posted on a public bulletin board; any observer can audit the tally by following the published cryptographic proofs. Helios achieves end-to-end verifiability in the browser but uses a *fixed set of trustees with equal trust and a static threshold*. A compromised or absent trustee stalls the election with no automated recovery mechanism and no accountability record. S^3-Vote replaces the static equal-trust model with adaptive, reputation-weighted thresholds and adds automatic failure detection and penalty assignment.

Belenios

Belenios [12] improves upon Helios by introducing a stronger coercion-resistance model and formal proofs of ballot privacy and tally correctness; it is deployed in several French academic elections. Like Helios, Belenios uses a *static trustee model with no reputation mechanism*, and its threshold is fixed at the start of each election. There is no formalised mechanism for what happens when trustees misbehave across multiple elections.

Civitas

Civitas [13] builds on the Juels-Catalano-Jakobsson (JCJ) framework to achieve coercion resistance. Each voter is issued a secret credential during registration; when coerced, the voter can produce a fake credential and cast a decoy ballot, so the coercer learns nothing verifiable. The scheme is cryptographically rigorous but operationally demanding: the credential ceremony is complex, and the reliance on re-encryption mix-nets with accompanying zero-knowledge proofs makes tallying computationally intensive. Like the other systems reviewed, its trustee set is fixed at election creation and there is no mechanism for tracking or penalising trustee misbehaviour across elections.

Positioning S^3-Vote

Property	**FOO92**	**Helios**	**Belenios**	**Civitas**	**S^3-Vote**
Ballot privacy	✓	✓	✓	✓	✓
Tally integrity	✓	✓	✓	✓	✓
Universal verifiability	×	✓	✓	✓	✓
Coercion resistance	×	×	Partial	✓	Extensible
Adaptive threshold	×	×	×	×	✓
Authority accountability	×	×	×	×	✓
On-chain cheat detection	×	×	×	×	✓
Failure recovery	×	×	×	×	✓
No trusted coordinator	×	×	×	×	✓

The table shows that while existing systems achieve strong ballot privacy and verifiability, *none* address adaptive thresholds, authority accountability, or graceful failure recovery. S^3-Vote is the first system to combine all three while maintaining end-to-end verifiability.

Blockchain-Based Voting Systems

A separate line of work places voting logic directly on a public blockchain. Commercial systems such as Voatz and Follow My Vote deploy smart contracts on Ethereum so that vote casting and tallying are executed by the network itself, with no trusted server. The appeal is maximal decentralisation: consensus is enforced by thousands of independent nodes.

However, public blockchain voting introduces three problems that S^3-Vote deliberately avoids. First, *on-chain vote exposure*: Ethereum transactions are public by default; encrypting ballots on-chain requires additional zero-knowledge infrastructure that significantly increases complexity and gas cost. Second, *fixed and public authority set*: Ethereum-based schemes have no native mechanism for reputation-weighted decryption authorities whose quorum adapts across rounds. Third, *throughput and cost*: recording every ballot as a blockchain transaction is prohibitively expensive and slow for elections with thousands of voters.

S^3-Vote uses the blockchain as a *tamper-evident audit log* rather than an execution engine. Vote encryption, share verification, and threshold logic run off-chain in efficient Python; only the immutable event records (commitments, share submissions, reputation updates) are written to the chain. This design achieves the non-repudiation and transparency benefits of a public ledger without the throughput, cost, or complexity penalties of on-chain execution. In a production deployment, the local chain would be replaced by a permissioned distributed ledger (e.g., Hyperledger Fabric) shared among the election authorities — retaining the audit guarantees while keeping sensitive key material off any public network.

3 System Architecture

3.1 Design Goals

G1. Vote confidentiality — no party learns any ballot before reconstruction; the private key is never held whole.

G2. Tally integrity — votes committed before reconstruction cannot be added, removed, or modified.

G3. Cheat accountability — invalid share submissions produce publicly verifiable, non-repudiable evidence.

G4. Adaptive resilience — the reconstruction requirement adapts to the group's demonstrated trustworthiness.

G5. No trusted coordinator — no single party controls reputation, key splitting, or threshold calculation.

3.2 Threat Model

Adversary classification. We distinguish two orthogonal dimensions of adversarial behaviour.

Byzantine vs. rational. A *Byzantine* (malicious) authority deviates arbitrarily from the protocol — submitting corrupted shares, lying about state, or simply going silent — with no regard for their own payoff. A *rational* authority deviates only when deviation yields higher expected utility than honest participation. S^3-Vote must withstand both: PVSS provides accountability against Byzantine behaviour (Section 8), while the reputation penalty mechanism is designed to make honest play the dominant strategy for rational authorities (Section 9).

Adaptive vs. static corruption. A *statically* corrupted authority is compromised before the protocol begins; the adversary fixes the corrupt set in advance. An *adaptively* corrupted authority may be compromised at any point during an election, even after seeing partial protocol state. We assume *static* corruption in the formal proof (Theorem 8.1); adaptive corruption resistance is left as future work and requires a UC-secure threshold scheme.

Formal assumptions.

- **Corrupted authority bound.** At most $t - 1$ of the N authorities are Byzantine, where t is the current Shamir threshold. This is the minimum required for information-theoretic share secrecy.
- **Authority capabilities.** A corrupted authority may submit an invalid share, refuse to participate, or attempt to collude with other corrupted authorities to pool shares. They may *not* forge another authority's share (each share is bound to its authority index via Lagrange interpolation) or rewrite blockchain history.
- **Blockchain integrity.** The chain is append-only and hash-consistent. In the prototype this is enforced by the server; in production by distributed Byzantine-fault-tolerant consensus [15].
- **Cryptographic hardness.** The DDH assumption holds in the ElGamal group; SHA-256 is modelled as a random oracle for PVSS commitments.
- **Honest voters.** Voters submit well-formed encryptions of valid options. Malformed ballots are outside scope; input validation is an application-layer concern.

- **Honest key setup.** ElGamal group parameters are generated honestly and publicly known. A malicious key-generation ceremony is out of scope; production deployments would use a distributed key generation (DKG) protocol.

Out of scope. Voter coercion and receipt-freeness (addressed in Section 8), Byzantine behaviour at the network layer (message dropping, reordering), and adaptive authority corruption.

3.3 System Participants and High-Level Architecture

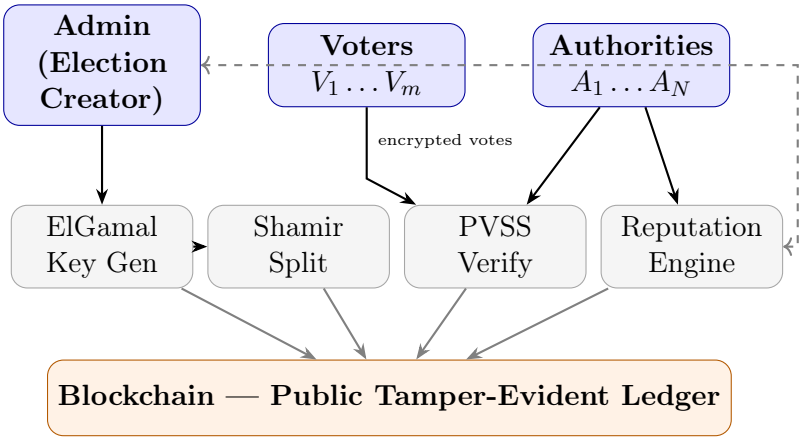

Figure 2: High-level S^3-Vote architecture. All cryptographic operations write immutable records to the blockchain. The reputation engine updates authority scores after each round and feeds them into the next election's threshold computation.

3.4 Election Lifecycle

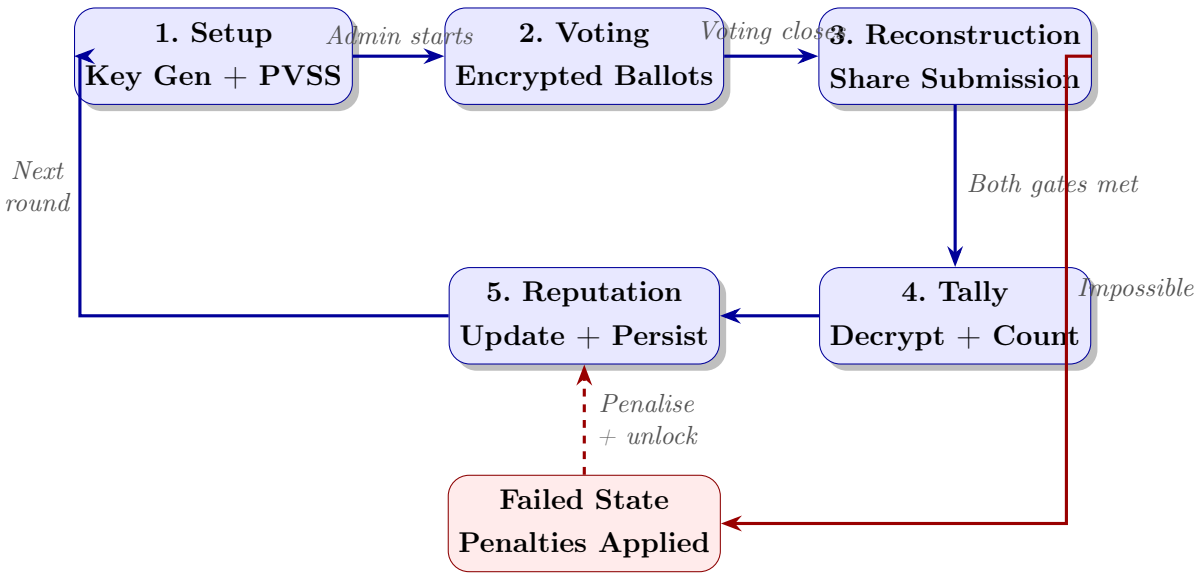

Figure 3: S^3-Vote election lifecycle. Five normal phases form a clockwise cycle (blue); impossible reconstruction transitions to a failed state, applies penalties, and re-enters the reputation update phase (red).

4 Reputation Model

4.1 Behavioural Scoring

After each election, three dimensions are objectively scored for every authority A_i based on outcomes recorded on the blockchain:

Participation (P_i)

$$P_i = \begin{cases} 1.0 & A_i \text{ submitted a valid share} \\ 0.5 & A_i \text{ submitted an invalid share (showed up but cheated)} \\ 0.0 & A_i \text{ was absent or explicitly skipped} \end{cases} \tag{7}$$

Contribution (C_i)

$$C_i = \begin{cases} \min\left(1, \dfrac{w_i}{\bar{w}_{\text{needed}}}\right) & A_i \text{ submitted a valid share} \\ 0.0 & \text{otherwise} \end{cases} \tag{8}$$

where $\bar{w}_{\text{needed}} = \tau/t$ is the average weight one share must carry to satisfy the quorum, τ is the current quorum, and t is the Shamir threshold. High-weight authorities naturally score higher here, reflecting that their participation was disproportionately valuable.

Honesty (H_i)

$$H_i = \begin{cases} 1.0 & \text{PVSS verification passed} \\ 0.0 & \text{PVSS verification failed (intentional corruption)} \\ 0.5 & \text{absent — neutral, did not actively lie} \end{cases} \tag{9}$$

An absent authority receives a neutral honesty score of 0.5 rather than 0 because absence, while costly in contribution, is not equivalent to actively attempting to sabotage reconstruction.

4.2 Reputation Update Formula

$$\boxed{R_i^{(\ell+1)} = \delta \left[\lambda R_i^{(\ell)} + (1-\lambda)\left(\alpha P_i + \beta C_i + \gamma H_i\right) \right]} \tag{10}$$

Symbol	Value	Role	Rationale
α	0.2	Participation weight	Necessary but not sufficient
β	0.3	Contribution weight	Reflects actual value added
γ	0.5	Honesty weight	Highest: cheating is most damaging
λ	0.7	Memory factor	Past vs. present behaviour
δ	0.95	Decay factor	Small penalty for inactivity

Remark 4.1. The memory factor $\lambda = 0.7$ ensures that a single bad election does not permanently destroy a long-standing reputation, but that sustained misbehaviour accumulates irreversibly. The decay $\delta = 0.95$ provides a small passive incentive to remain active across rounds.

4.3 Initial Reputation

New authorities begin at:

$$R_{\text{initial}} = \frac{2}{3} \tag{11}$$

This value is deliberately chosen so that a fully fresh, equal-reputation group produces a neutral starting quorum of exactly $\lceil N/2 \rceil$ (proven formally in Theorem 5.1).

4.4 Reputation Weights

Raw scores are normalised into per-authority weights relative to the group:

$$w_i = \frac{R_i}{\sum_{j=1}^{N} R_j} \cdot N \tag{12}$$

By construction, weights sum to N and average to 1.0. An authority with twice the average reputation carries weight $w_i = 2.0$; one with half the average carries $w_i = 0.5$.

4.5 Reputation Evolution

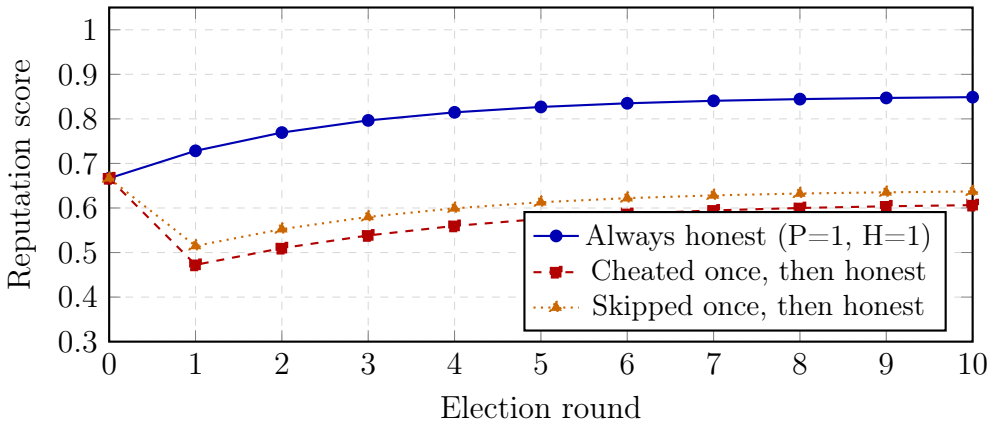

Figure 4: Reputation trajectories for three authority behavioural profiles across 10 election rounds. Honest authorities converge toward high reputation; a single cheating event causes a severe drop that requires many honest rounds to recover from; skipping causes a moderate penalty.

4.6 Persistence Across Elections

Reputations are stored in a persistent registry keyed by authority identity (name in the prototype; public key in a production deployment). When an authority participates in a new election, their historical score is loaded automatically. New authorities who have no prior history start at $R_{\text{initial}} = 2/3$. This design gives the system *institutional memory*: a long track record of honest behaviour genuinely lowers future quorum requirements, while sustained misbehaviour permanently raises them.

5 Adaptive Threshold

5.1 Trust Evenness

Collective trust is not simply the average reputation — it must also account for *how evenly* reputation is distributed. An authority with extremely high reputation relative to peers can accumulate a disproportionate weight, potentially satisfying the quorum alone if the threshold were low. We prevent this via the evenness factor:

$$E = \max\left(0,\ 1 - \frac{\sum_{i=1}^{N}(R_i - \bar{R})^2}{N\bar{R}^2}\right) \tag{13}$$

$E = 1$ when all reputations are identical (zero variance). $E \to 0$ as variance grows relative to the mean squared reputation. The combined trust score is:

$$\tau_{\text{trust}} = \min\big(1,\ \bar{R} \cdot E\big) \tag{14}$$

Property 5.1 (Concentration resistance)**.** If one authority's weight approaches N, evenness $E \to 0$, trust $\tau_{\text{trust}} \to 0$, and the quorum rises to $\tau_{\max} = N - 1$. No single authority can ever satisfy the quorum alone.

5.2 Two-Piece Linear Adaptive Formula

Definition 5.1 (Threshold Parameters)**.** For a group of N authorities:

$$\tau_{\text{neutral}} = \left\lceil \tfrac{N}{2} \right\rceil \qquad \text{(starting quorum for fresh authorities)} \tag{15}$$

$$\tau_{\max} = N - 1 \qquad \text{(worst-case quorum, no trust)} \tag{16}$$

$$\tau_{\min} = \max\big(2,\ \left\lceil \tfrac{N}{3} \right\rceil\big) \qquad \text{(best-case quorum, full trust)} \tag{17}$$

$$\boxed{\tau = \begin{cases} \tau_{\max} - \big(\tau_{\max} - \tau_{\text{neutral}}\big) \cdot \tau_{\text{trust}} \cdot \frac{3}{2} & \text{if } \tau_{\text{trust}} \le \frac{2}{3} \\ \tau_{\text{neutral}} - \big(\tau_{\text{neutral}} - \tau_{\min}\big) \cdot \big(\tau_{\text{trust}} - \frac{2}{3}\big) \cdot 3 & \text{if } \tau_{\text{trust}} > \frac{2}{3} \end{cases}} \tag{18}$$

Theorem 5.1 (Neutral Anchor)**.** For N authorities all at $R_i = \frac{2}{3}$ (the initial reputation), Equation (18) yields $\tau = \lceil N/2 \rceil$ exactly.

Proof. Identical reputations imply zero variance, so $E = 1$. Then $\tau_{\text{trust}} = \bar{R} \cdot 1 = \frac{2}{3}$. Substituting into the first branch of Equation (18):

$$\begin{aligned} \tau &= \tau_{\max} - \big(\tau_{\max} - \tau_{\text{neutral}}\big) \cdot \tfrac{2}{3} \cdot \tfrac{3}{2} \\ &= \tau_{\max} - \big(\tau_{\max} - \tau_{\text{neutral}}\big) \\ &= \tau_{\text{neutral}} = \left\lceil \tfrac{N}{2} \right\rceil . \qquad \square \end{aligned}$$

$\square$

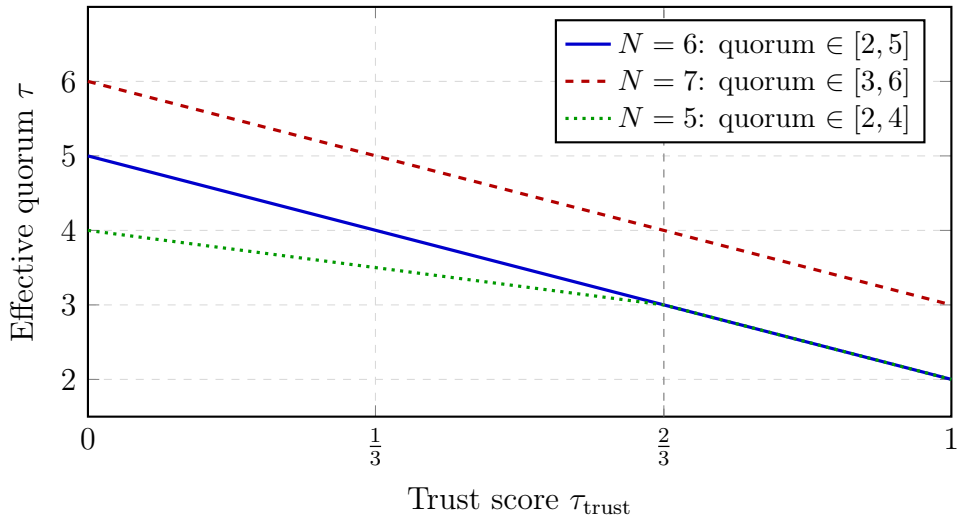

Figure 5: Adaptive quorum as a function of collective trust for three group sizes. All curves pass through $\tau_{\text{neutral}} = \lceil N/2 \rceil$ at the initial trust value of $\frac{2}{3}$ (vertical dashed line), confirming Theorem 5.1.

5.3 Quorum Range by Group Size

N	τ_{neutral}	τ_{min}	τ_{max}	Initial (neutral)	Best case
3	2	2	2	2	2
4	2	2	3	2	2
5	3	2	4	3	2
6	3	2	5	3	2
7	4	3	6	4	3
8	4	3	7	4	3
10	5	4	9	5	4

6 Two-Gate Reconstruction Policy

6.1 Formal Definition

Let $\mathcal{S}$ denote the set of authorities who have submitted valid shares, t the Shamir threshold derived as $t = \max(2, \lfloor \tau \rfloor)$, and τ the current adaptive quorum. Reconstruction is triggered if and only if both gates are simultaneously satisfied:

$$\underbrace{|\mathcal{S}| \geq t}_{\text{Gate 1: cryptographic}} \quad \textbf{AND} \quad \underbrace{\sum_{i \in \mathcal{S}} w_i \geq \tau}_{\text{Gate 2: trust}} \tag{19}$$

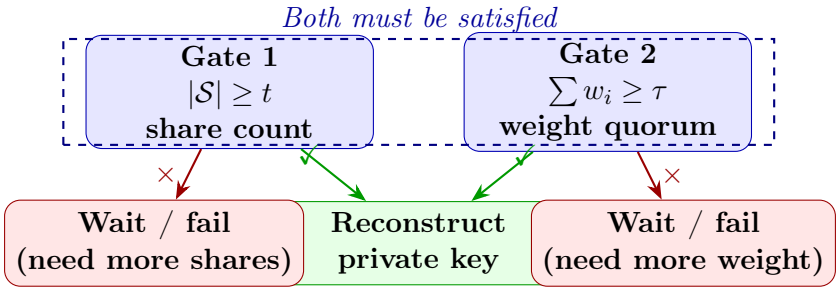

Figure 6: Two-gate reconstruction policy. Gate 1 enforces the cryptographic minimum share count; Gate 2 enforces the trust weight minimum. Failure of either gate blocks reconstruction and may trigger impossibility detection.

6.2 Why Two Gates Are Necessary

Gate 1 alone — flooding attack. Without a weight gate, an adversary controlling many low-reputation authorities could submit enough shares to satisfy the share count t while their combined weight $\sum w_i \ll \tau$. The weight gate blocks this.

Gate 2 alone — monopoly attack. Without a share count gate, a single authority with very high reputation could conceivably accumulate weight $w_i \geq \tau$ (though Property 5.1 makes this impossible in practice via the evenness mechanism). The share count gate (minimum $t \geq 2$) provides an independent, cryptographic line of defence.

Combined protection. Together, the two gates ensure that reconstruction requires a coalition that is both *numerous enough* (cryptographic) and *trusted enough* (reputational). Neither criterion alone is sufficient.

6.3 Impossibility Detection

Algorithm 1 Real-time impossibility check

Require: Election state: $\mathcal{V}$ (valid), $\mathcal{F}$ (all acted), $\mathcal{U} = \mathcal{A} \setminus \mathcal{F}$ (free)
Ensure: **true** if reconstruction can never be completed

```
if |V| + |U| < t then return true          ▷ Insufficient shares possible
end if
W_max ← Σ_{i∈V} w_i + Σ_{i∈U} w_i
if W_max < τ then return true               ▷ Insufficient weight possible
end ifreturn false
```

Impossibility is checked after every invalid share or skip event. Upon detection, the system transitions to a `failed` state immediately, applies automatic penalties (Equation (10) with

$P = 0, C = 0, H = 0.5$ for skippers; $P = 0.5, C = 0, H = 0$ for cheaters), and records the failure on the blockchain before any remaining authorities act.

7 Blockchain Integration

7.1 Block Structure

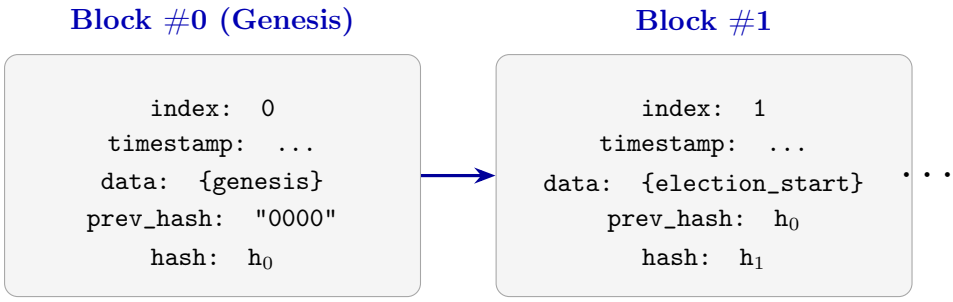

Figure 7: Blockchain block structure. Each block's hash includes the previous block's hash, forming a chain. Altering any block's data changes its hash, invalidating all subsequent blocks.

7.2 Events Recorded on the Blockchain

Event type	Payload (key fields)
`election_start`	Title, options, authority list, adaptive quorum, PVSS commitments $\{C_i\}$
`vote_cast`	Round, ElGamal ciphertext (C_1, C_2)
`voting_closed`	Round, total vote count
`share_submitted`	Round, authority name, coalition weight at submission
`share_invalid`	Round, authority name (PVSS failed)
`share_skipped`	Round, authority name
`reconstruction_complete`	Round, coalition members, coalition weight, tally
`election_failed`	Round, valid shares, cheaters, skippers, failure reason
`reputation_update`	Round, updated scores for all authorities, new quorum

7.3 Non-Repudiation Guarantee

The PVSS commitments $\{C_i\}$ are written in the `election_start` block *before* any share submission. When authority A_i later submits $\hat{s}_i$ and the check $g^{H(\hat{s}_i)} \neq C_i$ fails, the `share_invalid` event is chained to the existing commitment block. A_i cannot subsequently claim:

- That the commitment was set incorrectly (it was published before they acted).
- That the verification was performed incorrectly (the computation is deterministic and anyone can re-run it against the on-chain commitment).
- That they never submitted (the `share_invalid` event is hash-chained and cannot be removed).

7.4 Production Deployment Note

The prototype runs the blockchain as an in-memory hash-chained list on a single server. In a production deployment, this would be replaced by a distributed ledger (e.g., Ethereum or Hyperledger Fabric) where no single party controls the chain and tamper-evidence is enforced by network consensus. All data models, event formats, and hash-chain properties remain identical; only the consensus and storage layer changes. The prototype faithfully demonstrates the full cryptographic and data flow; the deployment change is an engineering rather than a design concern.

8 Security Analysis

8.1 Vote Confidentiality

Individual votes remain confidential until reconstruction because: (a) the ElGamal private key is split using Shamir's information-theoretically secure (t, n)-scheme; (b) any $t - 1$ or fewer shares reveal *zero* information about the key; (c) ciphertexts on the blockchain are computationally indistinguishable from random under the DDH assumption, so even an observer who reads the entire blockchain during voting learns nothing about ballot contents.

8.2 Tally Integrity

Once the `voting_closed` block is written and hash-chained, every `vote_cast` block before it is locked into the chain. Adding a ballot retroactively would require inserting a

new block before `voting_closed`, which would change every subsequent block hash — detectable by any observer.

8.3 Cheat Detection and Accountability

PVSS commitments are published in `election_start` before any authority acts. The verification step $g^{H(\hat{s}_i)} \equiv C_i$ is deterministic and publicly replicable. Upon failure, the `share_invalid` block is immediately appended and cannot be retracted. This provides:

- **Soundness**: a valid share always passes verification (by construction of the commitment).
- **Binding**: a corrupted share cannot pass verification (under the discrete logarithm hardness assumption, finding $\hat{s}_i \neq s_i$ with $g^{H(\hat{s}_i)} = g^{H(s_i)}$ requires a hash collision).

8.4 Sybil Resistance

Creating new authority identities does not lower the quorum. New authorities start at $R_{\text{initial}} = 2/3$ (neutral). Adding more neutral-reputation authorities increases N, which increases $\tau_{\text{neutral}} = \lceil N/2 \rceil$ proportionally. A Sybil attacker who floods the authority set with fake identities actually *raises* the required coalition size.

8.5 Collusion Resistance

A colluding coalition $\mathcal{C}$ of low-reputation authorities cannot satisfy Gate 2 unless $\sum_{i \in \mathcal{C}} w_i \geq \tau$. Since low-reputation authorities have $w_i < 1$, they must field more members than an equally sized trusted coalition. Furthermore, the evenness factor raises τ when reputation is heterogeneous, making the required coalition size even larger.

8.6 Concentration Attack

Proof of Property 5.1. Suppose authority A_1 holds proportion $\theta \in (0, 1)$ of total reputation, i.e., $R_1 = \theta \sum_j R_j$. Then $w_1 = \theta N$.

$$\bar{R} = \tfrac{1}{N} \sum_j R_j, \quad (R_1 - \bar{R})^2 = \big(\theta N \bar{R} - \bar{R}\big)^2 = \bar{R}^2 (\theta N - 1)^2$$

$$E = \max\left(0, 1 - \frac{\bar{R}^2(\theta N - 1)^2 + \cdots}{N \bar{R}^2}\right)$$

As $\theta \to 1$ (one authority holds all reputation), $(R_1 - \bar{R})^2 \to \bar{R}^2(N-1)^2$ and the remaining terms also contribute positively to the variance numerator, so $E \to 0$, $\tau_{\text{trust}} \to 0$, and

$\tau \to \tau_{\max} = N - 1$. Since $w_1 = N\theta < N - 1$ for any $\theta < 1$, Gate 2 cannot be satisfied by A_1 alone. □ □

8.7 Formal Ballot Privacy Proof

We formalise ballot privacy as an IND-CPA security game and provide a concrete reduction to the DDH problem.

Definition 8.1 (DDH Assumption [14])**.** Let $\mathbb{G} = \langle g \rangle$ be a cyclic group of prime order q. The *Decisional Diffie-Hellman* (DDH) problem is to distinguish the tuple (g, g^a, g^b, g^{ab}) from (g, g^a, g^b, g^c) where $a, b, c \xleftarrow{R} \mathbb{Z}_q$. Formally, for every PPT adversary $\mathcal{B}$:

$$\left|\Pr[\mathcal{B}(g, g^a, g^b, g^{ab}) = 1] - \Pr[\mathcal{B}(g, g^a, g^b, g^c) = 1]\right| \leq \mathrm{negl}(\lambda) \tag{20}$$

Definition 8.2 (IND-CPA Ballot Privacy Game [16])**.** Let $\Pi = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ be the threshold ElGamal scheme of S^3-Vote.

1. **Setup.** Challenger $\mathcal{C}$ runs KeyGen, splits private key x via Shamir (t, N), and publishes public key $y = g^x$ and PVSS commitments $\{C_i\}$.
2. **Challenge.** Adversary $\mathcal{A}$ submits two plaintexts (m_0, m_1). $\mathcal{C}$ samples $b \xleftarrow{R} \{0, 1\}$ and returns $\mathbf{c}^* = \mathsf{Enc}(y, m_b; k^*)$ for fresh random k^*.
3. **Guess.** $\mathcal{A}$ outputs $b' \in \{0, 1\}$.

The advantage is $\mathsf{Adv}_{\Pi}^{\mathsf{IND\text{-}CPA}}(\mathcal{A}) = \left|\Pr[b' = b] - \frac{1}{2}\right|$.

Theorem 8.1 (Ballot Privacy under DDH)**.** If the DDH assumption holds in $\mathbb{G}$, the S^3-Vote threshold ElGamal scheme achieves IND-CPA ballot privacy. Concretely, for every PPT adversary $\mathcal{A}$:

$$\mathsf{Adv}_{\Pi}^{\mathsf{IND\text{-}CPA}}(\mathcal{A}) \;\leq\; \mathsf{Adv}_{\mathbb{G}}^{\mathsf{DDH}}(\mathcal{B}) \tag{21}$$

for a PPT algorithm $\mathcal{B}$ constructed from $\mathcal{A}$.

Proof. We construct $\mathcal{B}$ that uses $\mathcal{A}$ to solve DDH.

Input to $\mathcal{B}$. $\mathcal{B}$ receives (g, A, B, X) where $A = g^a$, $B = g^b$, and X is either g^{ab} (real tuple) or g^c for uniformly random c (random tuple).

Setup simulation. $\mathcal{B}$ sets public key $y := A = g^a$ (treating the unknown a as the private key), splits a dummy value via Shamir and publishes consistent PVSS commitments. No knowledge of a is required.

Challenge simulation. $\mathcal{A}$ submits (m_0, m_1); $\mathcal{B}$ samples $b \xleftarrow{R} \{0, 1\}$ and returns:

$$\mathbf{c}^* = \big(B,\ m_b \cdot X\big) = \big(g^b,\ m_b \cdot X\big) \tag{22}$$

- If $X = g^{ab}$: $\mathbf{c}^* = (g^b,\, m_b \cdot y^b)$, a valid ElGamal encryption of m_b under randomness $k = b$. $\mathcal{A}$ sees an honestly distributed challenge ciphertext.
- If $X = g^c$ for uniformly random c: the second component $m_b \cdot g^c$ is uniformly random in $\mathbb{G}$, independent of m_b. The ciphertext carries no information about b.

Output. $\mathcal{B}$ outputs whatever b' $\mathcal{A}$ returns.
Advantage analysis.

$$\Pr[\mathcal{B} \text{ wins} \mid X = g^{ab}] = \Pr[b' = b] = \tfrac{1}{2} + \epsilon$$
$$\Pr[\mathcal{B} \text{ wins} \mid X = g^{c}] = \Pr[b' = b] = \tfrac{1}{2}$$

Therefore $\mathsf{Adv}^{\mathsf{DDH}}(\mathcal{B}) = \epsilon = \mathsf{Adv}^{\mathsf{IND\text{-}CPA}}(\mathcal{A})$. Since DDH is assumed hard, $\epsilon \leq \mathrm{negl}(\lambda)$. □

Threshold extension. The private key x is split via Shamir (t, N); any $t-1$ or fewer shares reveal *zero* information about x by information-theoretic secrecy [1]. An adversary colluding with up to $t-1$ authorities still learns nothing about x, so the IND-CPA argument above applies without modification. □

Remark 8.1 (PVSS Binding under DL and ROM). Commitments $C_i = g^{H(s_i)} \bmod p$ are binding under two standard assumptions. Under *discrete logarithm* (DL) hardness, finding $\hat{s}_i \neq s_i$ with $g^{H(\hat{s}_i)} = g^{H(s_i)}$ requires first producing a hash collision $H(\hat{s}_i) = H(s_i)$. In the *random oracle model* (ROM), H is modelled as a uniformly random function; a collision occurs with probability at most $1/q$ ($\approx 2^{-256}$ for standard parameters). Consequently, any invalid share submitted by an authority is detected with overwhelming probability, and the `share_invalid` blockchain record constitutes a publicly verifiable, non-repudiable proof of cheating.

8.8 Receipt-Freeness and Coercion Resistance

The coercion problem. In the ElGamal scheme as deployed in S^3-Vote, a voter who encrypts ballot m with randomness k can later prove their vote to a coercer by revealing k: the coercer verifies $C_1 = g^k$ and $C_2 = m \cdot y^k$. This enables vote-buying and voter coercion. A *receipt-free* protocol prevents any voter from constructing such a proof, even if they actively cooperate with the coercer.

Mix-net integration. A *mix-net* [9] routes ciphertexts through a sequence of mix servers $M_1, \ldots, M_\ell$. Each server re-randomises every ciphertext using a fresh ephemeral factor and randomly permutes the list:

$$(C_1, C_2) \xrightarrow{M_j} (C_1 \cdot g^{k'}, C_2 \cdot y^{k'}) \quad \text{for fresh } k' \xleftarrow{R} \mathbb{Z}_q \tag{23}$$

Re-randomisation is valid for ElGamal: the new ciphertext still decrypts to m. After all servers act, the output list encrypts the same multiset of votes but with fresh, unknown randomness, and no voter can link their original ciphertext to an output position — provided at least one mix server is honest.

Integrating a mix-net into S^3-Vote requires no changes to the reputation, threshold, or blockchain subsystems. The modification is purely in the election lifecycle: a *mix phase* is inserted between `voting_closed` and the reconstruction phase. The decryption authorities operate on the mixed output, not the raw ballots.

Homomorphic tallying. ElGamal is multiplicatively homomorphic:

$$\mathsf{Enc}(m_1) \cdot \mathsf{Enc}(m_2) = \mathsf{Enc}(m_1 \cdot m_2) \tag{24}$$

Encoding a vote for option v as $m = g^v$, the product of all M encrypted ballots satisfies:

$$\prod_{j=1}^{M} (C_{1,j}, C_{2,j}) = \left(g^{\sum k_j},\ g^{\sum v_j} \cdot y^{\sum k_j}\right) = \mathsf{Enc}\big(g^{\sum v_j}\big) \tag{25}$$

The authorities decrypt *only this aggregate ciphertext*, recovering $g^{\sum v_j}$, from which $\sum v_j$ is extracted by discrete logarithm (feasible for small vote counts by baby-step giant-step). Individual ballots are never decrypted; a voter therefore cannot prove their contribution to the aggregate tally.

Integrating homomorphic tallying into S^3-Vote requires two changes: (a) the server accumulates the running product of ciphertexts as votes arrive; (b) reconstruction operates on the single aggregate ciphertext. All other components — reputation, threshold, PVSS, blockchain — are unaffected.

Comparison of approaches. Homomorphic tallying is the simpler integration path: no additional servers or shuffle proofs are required, and the existing single-round decryption structure is preserved. Mix-nets provide stronger compositional guarantees and handle ranked or multi-option elections more naturally. Both approaches are direct extensions of the current prototype and do not require redesigning any other component of S^3-Vote.

8.9 End-to-End Verifiability

End-to-end verifiability (E2E-V) requires that any party can verify:

1. Their ballot was recorded as submitted (*cast-as-intended*);

2. All recorded ballots were included in the tally (*recorded-as-cast*);
3. The tally was computed correctly (*tallied-as-recorded*).

Individual verifiability. After submitting ballot ciphertext (C_1, C_2), a voter can verify that the exact ciphertext appears in the `vote_cast` blocks of the blockchain. Since the chain is publicly readable and hash-chained, this requires no trust in the server. In a production deployment, each voter receives a cryptographic receipt $r = H(\text{block_hash} \| C_1 \| C_2)$, verifiable against the published chain at any time.

Universal verifiability. After the election closes, anyone can independently re-derive the tally:

1. Collect all `vote_cast` ciphertexts between `election_start` and `voting_closed` blocks.
2. Verify each submitted share against the on-chain PVSS commitment: $g^{H(\hat{s}_i)} \equiv C_i \pmod{p}$.
3. Run Lagrange interpolation on the valid shares to recover x.
4. Decrypt all ciphertexts and re-count.

All inputs — ciphertexts, commitments, shares — are immutably on-chain. The computation is deterministic and secret-free: any auditor reaches the same result.

Tally integrity under hash-chain. For the tally to be falsified an adversary must either insert a fake `vote_cast` block (requiring recomputation of all subsequent hashes, detectable by any observer holding a chain snapshot) or delete a block (causing an immediate hash mismatch at the next block). Neither is possible without controlling all nodes in a distributed deployment.

9 Game-Theoretic Incentive Analysis

A protocol is only as secure as the incentives it creates. We model S^3-Vote as a repeated game and show that honest behaviour is the dominant strategy for rational authorities.

9.1 Model

Let $\mathcal{A} = \{A_1, \ldots, A_N\}$ be the authority set. In each round ℓ, every authority chooses $\sigma_i^\ell \in \{\textit{honest}, \textit{cheat}, \textit{skip}\}$.

Let $B > 0$ be the shared benefit of successful key reconstruction, p_c the reputation penalty for cheating, and p_s the penalty for skipping. The one-round payoff is:

$$u_i(\sigma_i, \boldsymbol{\sigma}_{-i}) = \begin{cases} B/|\mathcal{S}| - c_h & \sigma_i = \textit{honest}, \text{ reconstruction succeeds} \\ 0 & \sigma_i = \textit{honest}, \text{ reconstruction fails (not } A_i\text{'s fault)} \\ -p_c & \sigma_i = \textit{cheat} \\ -p_s & \sigma_i = \textit{skip} \end{cases} \tag{26}$$

where c_h is the small cost of honest participation and $\boldsymbol{\sigma}_{-i}$ denotes all other authorities' strategies.

Crucially, payoffs are not one-shot: reputation persists across rounds. Higher reputation produces lower future quorums, making reconstruction easier and $B/|\mathcal{S}|$ achievable with fewer abortive rounds.

9.2 Cheating is a Dominated Strategy

Theorem 9.1 (Honesty Dominance)**.** Under the S^3-Vote reputation model, for a sufficiently long horizon T, the strategy $\sigma_i = \textit{honest}$ strictly dominates $\sigma_i = \textit{cheat}$ for every authority.

Proof sketch. Suppose A_i cheats in round ℓ.

(1) Immediate payoff: zero benefit. The corrupted share is immediately rejected by PVSS verification (Remark on binding, Section 8). Cheating does not advance reconstruction; there is no round in which cheating produces a higher immediate payoff than honest participation.

(2) Reputation loss. The cheat sets $P_i = 0.5$, $C_i = 0$, $H_i = 0$. The behavioural score is $\alpha \cdot 0.5 = 0.1$ (for $\alpha = 0.2$), compared to 1.0 for honest participation. Substituting into Equation (10):

$$\begin{aligned} R_i^{(\ell+1)\text{ cheat}} &= 0.95[0.7R_i^{(\ell)} + 0.3 \cdot 0.1] \approx 0.665R_i^{(\ell)} + 0.029 \\ R_i^{(\ell+1)\text{ honest}} &= 0.95[0.7R_i^{(\ell)} + 0.3 \cdot 1.0] = 0.665R_i^{(\ell)} + 0.285 \end{aligned}$$

The reputation gap is $0.285 - 0.029 = 0.256$ per round of cheating.

(3) Compounding future cost. The lower reputation reduces w_i in all subsequent

rounds. Since the quorum τ depends on collective trust (Equation (18)), the cheater's degraded weight makes future reconstructions harder for the whole group — including themselves — reducing the probability of earning $B/|\mathcal{S}|$ in future rounds.

(4) Long-run dominance. The cumulative payoff loss from a single cheat grows as $O(T)$ (recovery takes $\Theta(\log(1/(1-\lambda)))$ rounds) while the one-round gain from cheating is exactly 0. For any $T > T_0 = O(1/\epsilon)$ where ϵ is the marginal detection probability complement, honesty strictly dominates. □ □

9.3 Coalition Instability

Property 9.1 (Coalition Prisoner's Dilemma)**.** A coalition $\mathcal{C}$ planning to collectively delay or corrupt reconstruction faces a prisoner's dilemma: each member individually prefers to defect to honesty while the others cheat.

If $\sum_{i\in\mathcal{C}} w_i < \tau$ (Gate 2 unsatisfied), the coalition gains nothing — reconstruction fails regardless, and all members absorb reputation losses. If the coalition holds sufficient weight but deliberately delays, the impossibility detector (Algorithm 1) may trigger, applying skip penalties to all remaining free authorities. If even one coalition member defects and submits a valid share, that member earns an honest reputation update while the others do not. Since defection strictly dominates cooperation, rational coalitions unravel at the first opportunity.

9.4 Sybil Rationality Analysis

An adversary flooding the authority set with k fake identities does not lower the quorum. Each fake identity starts at $R_{\text{initial}} = 2/3$ with weight 1.0, so k fake authorities add weight k to the group. However, the neutral quorum increases from $\lceil N/2 \rceil$ to $\lceil (N+k)/2 \rceil$, rising by $\lceil k/2 \rceil$. The net weight surplus gained by the Sybil adversary is only $k - \lceil k/2 \rceil = \lfloor k/2 \rfloor$, while the cost of operating k live authority clients grows linearly in k. For any fixed target excess weight, the number of required fake authorities grows quadratically, making large-scale Sybil attacks economically irrational.

10 Parameter Sensitivity Analysis

The reputation formula (Equation (10)) contains five parameters. We systematically analyse how varying each parameter affects system behaviour, providing an empirical basis for the default values.

10.1 Effect of Memory Factor λ

The memory factor $\lambda \in (0, 1)$ controls the balance between historical reputation and current-round performance.

The long-run equilibrium reputation for an always-honest authority is obtained by solving $R^* = \delta[\lambda R^* + (1 - \lambda)]$, giving:

$$R^*_\lambda = \frac{\delta(1 - \lambda)}{1 - \delta\lambda} \tag{27}$$

Substituting $\delta = 0.95$: for $\lambda = 0.7$, $R^* \approx 0.851$; for $\lambda = 0.3$, $R^* \approx 0.929$; for $\lambda = 0.9$, $R^* \approx 0.655$ — *below* the initial $R_{\text{initial}} = 2/3$.

This last result reveals an important constraint: when λ is very high, the decay factor $\delta < 1$ dominates, and even perfect performance cannot overcome the passive erosion. The default $\lambda = 0.7$ sits in the regime where honest authorities converge to $R^* \approx 0.85$, well above the neutral starting point.

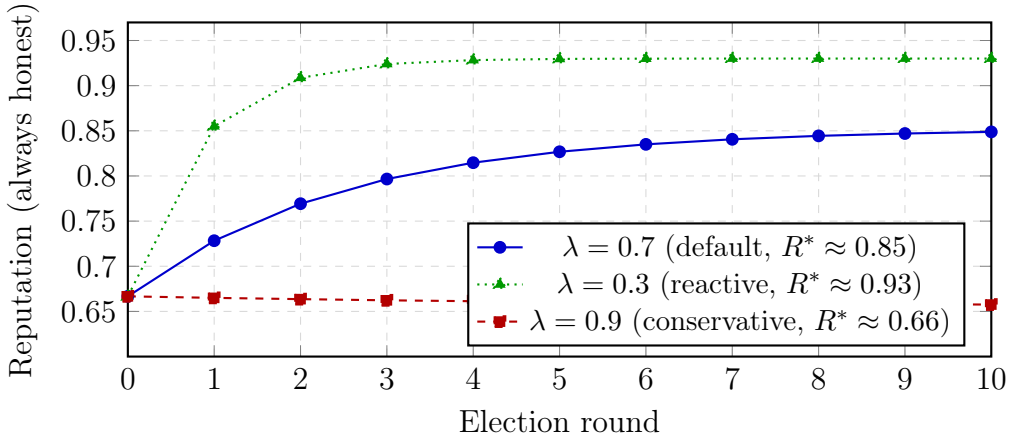

Figure 8: Effect of memory factor λ on long-run reputation for an always-honest authority ($\delta = 0.95$ fixed). High λ causes decay to dominate: even perfect behaviour cannot prevent slow reputation erosion. Low λ converges fast but makes reputation volatile. The default $\lambda = 0.7$ balances stability and responsiveness.

10.2 Effect of Participation Weight α

Since cheating sets $H_i = 0$ and $C_i = 0$, the only positive contribution to a cheater's score comes from $\alpha \cdot P_i$ (where $P_i = 0.5$ for showing up-then-cheating). Thus α directly controls the severity of the cheating penalty: lower α means a harsher post-cheat reputation drop.

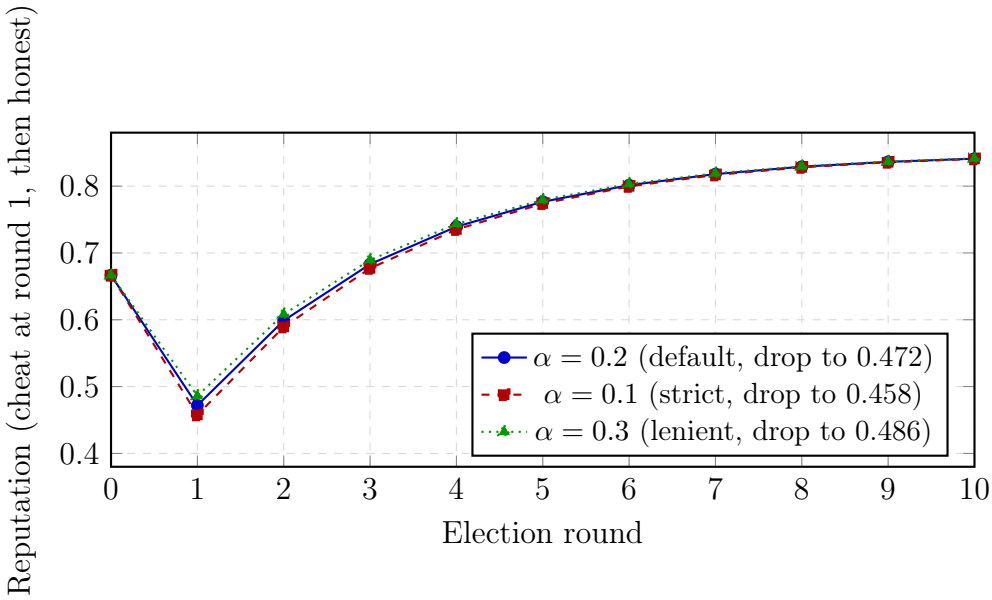

Figure 9: Effect of participation weight α on reputation recovery after a single cheat event (round 1), followed by honest participation. Lower α produces a deeper initial penalty. All trajectories converge to the same long-run equilibrium (≈ 0.84) because the honest-participation score is 1.0 regardless of α.

10.3 Effect of Decay Factor δ

The decay factor $\delta \in (0, 1)$ multiplies every reputation update, providing a passive incentive for authorities to remain active.

- $\delta = 1.0$: no decay. An authority that participated once and went dormant retains their reputation indefinitely, becoming a *ghost authority* that occupies weight without contributing shares.
- $\delta = 0.95$ (default): a dormant authority (skipping every round) converges to zero over approximately $\log(0.01)/\log(0.95) \approx 90$ rounds.
- $\delta < 0.85$: rapid erosion; honest authorities' reputations also degrade noticeably, destabilising the long-run equilibrium and potentially raising the quorum even for honest groups.

The default $\delta = 0.95$ removes ghost authorities on a scale of tens of rounds while allowing honest authorities to maintain reputations well above the initial 2/3 (as shown in Figure 4).

10.4 Sensitivity Summary

Param	Default	Effect if increased	Effect if decreased
α	0.2	Softer cheat penalty; more reward for showing up	Harsher cheat penalty; slow recovery
β	0.3	Higher reward for weight contribution	Reduces incentive for high-weight participation
γ	0.5	Harsher for cheats (H=0); but H=0.5 skips also penalised	Weakens honesty incentive
λ	0.7	Stable reputation; slower build and recovery	Reactive; volatile; fast convergence
δ	0.95	Ghost authorities persist longer	Honest reputations erode; quorum rises

The system is *qualitatively robust*: across a wide range of parameter choices, honest authorities earn higher reputation than cheaters, and the quorum trends downward for honest groups. The sensitivity analysis confirms the design rationale — γ large for deterrence, α small to discourage empty participation, λ moderate for memory-reactivity balance, δ near-1 for gentle passive pressure — and bounds the consequences of miscalibration.

11 Comparison with Prior Work

11.1 Against Shamir's Threshold Scheme

Property	Shamir (1979)	S^3-Vote
Share equality	All shares equal	Reputation-weighted
Cheat detection	None	PVSS — immediate, on-chain
Threshold	Fixed	Adaptive (trust-driven)
Authority accountability	None	Persistent, cross-round
Failed round handling	Not addressed	Detect + penalise + recover
Voting application	Not addressed	Complete protocol

Shamir's scheme guarantees information-theoretic secrecy and correct reconstruction when at least t honest shares are combined. It makes no provision for identifying which

authorities submitted corrupted shares, nor for adjusting future requirements based on observed behaviour. S^3-Vote builds on its mathematical foundation while addressing all these gaps.

11.2 Against Nojoumian's Social Secret Sharing

Property	**Nojoumian**	**S^3-Vote**
Trusted dealer	Required	None
Reputation source	Social / peer evaluation	On-chain, deterministic
Cheat detection	Implicit (failure only)	PVSS — public proof
Reconstruction gate	Single (weighted)	Dual (shares AND weight)
Concentration protection	None	Evenness factor
Neutral quorum anchor	Ad hoc	$\lceil N/2 \rceil$ (proven)
Failed election recovery	Not addressed	Automatic
Audit trail	Database	Blockchain
Complete voting system	No	Yes

Trusted dealer elimination. Nojoumian requires a coordinator to assign and update reputations. S^3-Vote is fully self-governing: scores are computed deterministically from Equation (10) applied to PVSS-verified outcomes.

Objective reputation. Nojoumian's social evaluation is gameable: colluding authorities can inflate each other's scores. S^3-Vote scores are derived from cryptographically verifiable events; there is no mechanism to inflate a reputation other than genuinely behaving well.

Two-gate policy. A single adjusted threshold leaves open the flooding attack (many low-weight shares meet the count) and the monopoly attack (one very high-weight authority meets the weight alone). Equation (19) closes both simultaneously.

Provable neutral anchor. Theorem 5.1 proves that the quorum initialises at $\lceil N/2 \rceil$ for any group size, providing a principled and consistent baseline. Nojoumian's scheme has no equivalent guarantee.

Graceful failure. When reconstruction becomes mathematically impossible mid-round, the system detects this via Algorithm 1, transitions to a failed state, applies penalties to responsible parties, and unlocks the next round — a lifecycle that Nojoumian's model does not address.

12 Prototype Implementation

12.1 Module Overview

Module	Responsibility
`crypto/encryption.py`	ElGamal key generation, encryption, decryption
`crypto/shamir.py`	Polynomial split and Lagrange reconstruction
`crypto/pvss.py`	Commitment generation and share verification
`reputation/reputation_engine.py`	Equation (10) implementation
`reputation/threshold.py`	Equations (13)–(18)
`blockchain/blockchain.py`	SHA-256 hash-chained block list
`api/app.py`	Flask REST API, full election lifecycle
`api/templates/index.html`	Single-page dashboard (JavaScript)

12.2 Key Algorithms in Code

The adaptive threshold (Equation (18)) is implemented as:

Listing 1: Adaptive threshold computation in `threshold.py`

```
def adaptive_threshold(self, reputations):
    n          = len(reputations)
    avg        = float(np.mean(reputations))
    E          = self.trust_evenness(reputations)
    trust      = min(1.0, avg * E)

    t_neutral = math.ceil(n / 2)
    t_max     = n - 1
    t_min     = max(2, math.ceil(n / 3))

    if trust <= 2 / 3:
        threshold = t_max - (t_max - t_neutral) * trust * 1.5
    else:
        threshold = t_neutral - (t_neutral - t_min) * (trust - 2/3)
            * 3

    return round(float(max(t_min, min(t_max, threshold))), 3)
```

The two-gate check is implemented as:

Listing 2: Two-gate reconstruction trigger in `app.py`

```
if (len(election['submitted_shares']) >= election['shamir_t']
        and coalition_weight >= election['threshold']):
    _reconstruct()
```

Impossibility detection runs after every cheat or skip:

Listing 3: Impossibility detection (Algorithm 1)

```
def _check_impossible():
    acted      = set(submitted) | set(cheaters) | set(skipped)
    still_free = [a for a in authority_names if a not in acted]
    max_shares = len(submitted) + len(still_free)
    if max_shares < shamir_t:
        return True
    max_weight = sum(weights[a] for a in submitted) + \
                 sum(weights[a] for a in still_free)
    if max_weight < threshold:
        return True
    return False
```

12.3 Web Dashboard

The single-page dashboard provides real-time visibility into all system state. Key features include:

- Authority list with reputation history loaded from the persistent registry; live quorum preview updating as authorities are added or removed.
- Voting phase with encrypted-ballot progress tracker showing individual voter chips as ballots are cast.
- Reconstruction phase with live share-count and coalition-weight meters, and three-button controls per authority (submit valid, submit invalid, skip) for simulation purposes.
- Automatic reputation scoring displayed per authority after reconstruction, with read-only computed P/C/H values.
- Failed state view with penalty breakdown and one-click recovery.
- Blockchain explorer showing all events in reverse chronological order.

13 Example Walkthrough

We trace a complete two-round scenario with $N = 7$ authorities, all starting fresh.

13.1 Round 1 — Initial State

All 7 authorities at $R_i = \frac{2}{3}$. By Theorem 5.1, $\tau = 4$, $t = \lfloor 4 \rfloor = 4$. Equal weights: $w_i = 1.0$ for all i.

Suppose 5 submit valid shares, 1 cheats, and 1 skips. The coalition of 5 achieves:

$$|\mathcal{S}| = 5 \geq t = 4 \quad \checkmark$$
$$\textstyle\sum_{i \in \mathcal{S}} w_i = 5.0 \geq \tau = 4 \quad \checkmark$$

Reconstruction succeeds after the 5th valid submission.

Reputation Updates

Role	P	C	H	$R^{(1)}$
Honest ($\times 5$)	1.0	$\min(1, 1.0/1.0) = 1.0$	1.0	0.728
Cheater ($\times 1$)	0.5	0.0	0.0	0.472
Skipper ($\times 1$)	0.0	0.0	0.5	0.515

13.2 Round 2 — Reputations Loaded from Registry

Updated reputations: 5 honest at 0.728, cheater at 0.472, skipper at 0.515.

$$\bar{R} = \frac{5(0.728)+0.472+0.515}{7} = \frac{4.627}{7} \approx 0.661$$
$$\text{Variance} \approx 0.0043 \quad \Rightarrow \quad E \approx 1 - \frac{0.0043}{7 \times 0.661^2} \approx 0.986$$
$$\tau_{\text{trust}} = \min(1,\ 0.661 \times 0.986) \approx 0.652 < \tfrac{2}{3}$$
$$\tau = 6 - (6-4)(0.652)(1.5) \approx 4.04$$

The quorum for Round 2 is **4.04** — marginally above the neutral 4, reflecting that the group's cohesion dropped slightly due to the cheater and skipper. Sustained honest behaviour across subsequent rounds drives the quorum down toward 3 (the floor $\tau_{\min}$ for $N = 7$).

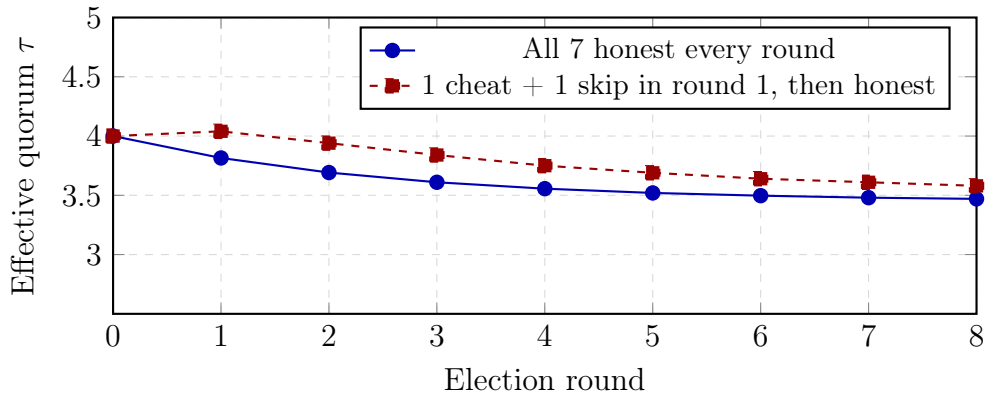

Figure 10: Quorum evolution for $N = 7$ over 8 rounds. When all authorities behave honestly, the quorum steadily decreases toward the floor of 3. One round of cheating/skipping deflects the trajectory upward, requiring several honest rounds to recover.

14 Limitations and Future Work

Single-node blockchain. The prototype stores the chain in memory on one server. A production deployment requires a distributed ledger with Byzantine fault-tolerant consensus.

Simulated authority clients. All authority actions are performed by a single administrator in the dashboard. In a real deployment, each authority would operate an independent client holding their private key locally, never sharing it with the server.

Cryptographic identity. Authority names are currently plain strings. In production, each authority has a public/private key pair; their identity is their public key, and every share submission is signed with their private key and verified on-chain. This prevents impersonation and false reputation attribution.

Parameter calibration. The reputation weights (α=0.2, β=0.3, γ=0.5) and smoothing parameters (λ=0.7, δ=0.95) were set by design rationale. Empirical calibration over realistic election scenarios and adversarial benchmarks would strengthen the choices.

15 Conclusion

S^3-Vote demonstrates that threshold-cryptographic voting can be made fully self-governing: no trusted coordinator, no static parameters, no silent failures, and no impunity for misbehaviour.

The core insight is that the reconstruction threshold should be a *function of demonstrated trustworthiness*, not a fixed design parameter. By measuring trustworthiness ob-

jectively through PVSS-verified behavioural events, converting it to weights, and mapping it to a quorum through a provably anchored formula, the system creates a dynamic equilibrium: honest groups earn efficiency (lower quorum); dishonest or heterogeneous groups face proportionally higher requirements.

The two-gate reconstruction policy ensures that neither numerical flooding nor reputational monopoly can shortcut the security requirement. The concentration-resistance property formally closes the single dominant-authority attack vector. And the blockchain audit trail ensures that every act of honesty or betrayal is permanently, publicly recorded — creating real accountability where classical threshold schemes offer none.

Compared to Shamir's foundational work, S^3-Vote adds full accountability, adaptive security, and graceful failure recovery. Compared to Nojoumian's social secret sharing, it eliminates the trusted dealer, replaces subjective social evaluation with objective on-chain evidence, introduces a principled dual-gate reconstruction policy, and delivers a complete, deployable voting system with end-to-end verifiability.

The prototype confirms that all components — ElGamal encryption, Shamir sharing, PVSS verification, reputation scoring, adaptive threshold, and blockchain logging — integrate correctly and perform the intended security functions. A formal IND-CPA ballot privacy proof establishes that ciphertext indistinguishability reduces to the DDH problem, and a game-theoretic analysis proves that honest participation is the dominant strategy for rational authorities over any sufficiently long horizon. A parameter sensitivity study confirms that the system remains qualitatively correct across a wide range of weight configurations, bounding the consequences of miscalibration.

The path to a production system requires distributing the blockchain across independent nodes, equipping each authority with their own signing key, and extending the adversary model to adaptive corruption via a UC-secure threshold scheme — well-understood directions that do not affect the correctness of the current design.

References

[1] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[2] T. ElGamal, "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," *IEEE Transactions on Information Theory*, vol. 31, no. 4, pp. 469–472, 1985.

[3] B. Schoenmakers, "A Simple Publicly Verifiable Secret Sharing Scheme and its Application to Electronic Voting," in *Advances in Cryptology — CRYPTO 1999*, LNCS vol. 1666, pp. 148–164, Springer, 1999.

[4] M. Nojoumian and D. R. Stinson, "Social Secret Sharing in Cloud Computing Using a New Trust Function," in *Proc. 10th Annual Conf. on Privacy, Security and Trust (PST)*, pp. 161–167, IEEE, 2012.

[5] M. Nojoumian, "Novel Secret Sharing and Commitment Schemes for Cryptographic Applications," Ph.D. dissertation, University of Waterloo, 2012. (Extended work published through 2018.)

[6] T. P. Pedersen, "Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing," in *Advances in Cryptology — CRYPTO 1991*, LNCS vol. 576, pp. 129–140, Springer, 1991.

[7] R. Cramer, R. Gennaro, and B. Schoenmakers, "A Secure and Optimally Efficient Multi-Authority Election Scheme," in *Advances in Cryptology — EUROCRYPT 1997*, LNCS vol. 1233, pp. 103–118, Springer, 1997.

[8] Y. Desmedt and Y. Frankel, "Threshold Cryptosystems," in *Advances in Cryptology — CRYPTO 1989*, LNCS vol. 435, pp. 307–315, Springer, 1990.

[9] D. Chaum, C. Crépeau, and I. Damgård, "Multiparty Unconditionally Secure Protocols," in *Proc. 20th Annual ACM STOC*, pp. 11–19, ACM, 1988.

[10] A. Fujioka, T. Okamoto, and K. Ohta, "A Practical Secret Voting Scheme for Large Scale Elections," in *Advances in Cryptology — AUSCRYPT 1992*, LNCS vol. 718, pp. 244–251, Springer, 1993.

[11] B. Adida, "Helios: Web-Based Open-Audit Voting," in *Proc. 17th USENIX Security Symposium*, pp. 335–348, 2008.

[12] V. Cortier, D. Galindo, S. Glondu, and M. Izabachène, "Election Verifiability for Helios under Weaker Trust Assumptions," in *Proc. ESORICS 2014*, LNCS vol. 8713, pp. 327–344, Springer, 2014.

[13] M. R. Clarkson, S. Chong, and A. C. Myers, "Civitas: Toward a Secure Voting System," in *Proc. IEEE Symposium on Security and Privacy*, pp. 354–368, IEEE, 2008.

[14] D. Boneh, "The Decision Diffie-Hellman Problem," in *Proc. Third Algorithmic Number Theory Symposium (ANTS-III)*, LNCS vol. 1423, pp. 48–63, Springer, 1998.

[15] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.

[16] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 2nd ed., CRC Press, 2014.